Technical Report - Book Metadata and Recommendation API

GitHub Repository: https://github.com/qt0713/Web-cwk

Repository review note: this report corresponds to the publicly accessible repository above. Examiners can review implementation details, commit history, and supporting files directly from that URL.

1. Project Overview

This project implements a RESTful Book Metadata and Recommendation API designed for coursework assessment in backend engineering, API design, and reflective practice.

The system supports authenticated CRUD operations for books, recommendation retrieval, and analytics endpoints. It also integrates a public dataset import workflow to demonstrate practical data engineering within a relational-database API.

Project objectives

- Build a runnable API with database-backed CRUD functionality.
- Implement secure access control using token-based authentication.
- Provide recommendation and analytics capabilities beyond minimum requirements.
- Demonstrate reproducible data import from a public dataset source.
- Produce supporting materials for oral demonstration and technical defence.

2. Technology Choices and Rationale

Programming language and framework

Python with FastAPI was selected for rapid endpoint development, strong typing support, and automatic OpenAPI documentation. This combination reduces boilerplate while preserving explicit API contracts.

Database

The project is designed around SQL databases:
- PostgreSQL as the production-oriented target.
- SQLite as a local development and testing option for fast setup and demonstrability.

This dual setup improves portability while staying aligned with module requirements for SQL-backed systems.

ORM and data access

SQLAlchemy was chosen for:
- clear relational modelling,
- controlled persistence logic,
- compatibility across SQLite and PostgreSQL.

Testing stack
Pytest with FastAPI TestClient was used for endpoint-level integration tests to validate real request/response behavior.

3. Architecture and Design Decisions

API style
The application uses RESTful resource-oriented routing under /api/v1, with separate route groups for authentication, books, and analytics.

Core data model
Primary entity: Book

Key fields:
- title
- author
- genre
- published_year
- average_rating
- ratings_count
- description
- created_at
- updated_at

Recommendation strategy
A lightweight heuristic strategy is used:
1. Prioritize books in the same genre as the seed book.
2. Rank by average_rating, then ratings_count.
3. If not enough items exist, fill with globally top-ranked books.

This approach is intentionally interpretable and suitable for oral explanation within coursework constraints.

Security model
JWT-based authentication is implemented via:
- register endpoint,
- login endpoint,
- bearer-token protection on book and analytics endpoints.

This provides a clear and assessable security improvement compared with unprotected CRUD.

Error handling
The API returns standard status codes with explicit error messages, including:
- 200, 201, 204 (success flows)
- 401 (authentication failure)
- 404 (resource not found)

- 409 (duplicate registration)
- 422 (validation failure)

4. Testing Approach and Results

Scope
Integration-level tests cover:
- health endpoint,
- authentication flow usage in protected routes,
- CRUD behavior,
- recommendation endpoint,
- analytics endpoint accessibility.

Method
- Pytest + FastAPI TestClient
- isolated SQLite test database per run

Result summary
Current automated test status: all tests pass.

This confirms that core API flows are functional after the latest data-import and analytics enhancements.

5. Public Dataset Integration

Dataset source
- Kaggle dataset: arashnic/book-recommendation-dataset
- Files used: Books.csv and Ratings.csv

Import implementation
A dedicated import script was developed to:
- ingest book metadata,
- handle CSV encoding/format inconsistencies,
- support ZIP-encapsulated CSV edge cases,
- merge rating statistics from ratings data using ISBN mapping.

Executed import record
- Command:
- python scripts/import_kaggle_books.py --csv-path .\Books.csv --ratings-csv-path .\Ratings.csv --limit 1000 --replace
- Result:
- Inserted = 1000
- Skipped = 0
- Verification:
- Database book count = 1000

6. Analytics Validation

After import, analytics endpoints were executed successfully and returned HTTP 200.

Observed behavior:
- /api/v1/analytics/genres reported a total count consistent with imported volume.
- /api/v1/analytics/ratings reported non-zero rating statistics and top-book ranking output.

Rating scale note
The imported ratings source uses a 0-10 scale. As a result, analytics averages can exceed 5.0. This is expected under current implementation and should be interpreted as source-scale preservation rather than normalization error.

7. Challenges and Resolutions

Challenge 1: dataset ingestion reliability
Real-world CSV files included encoding and packaging irregularities (for example, embedded ZIP content and parsing anomalies).

Resolution:
- strengthened importer robustness,
- added fallback parsing and extraction logic,
- separated download and local-file import pathways.

Challenge 2: balancing complexity with demonstrability
A fully personalized recommender would require significantly broader user-behavior modelling.

Resolution:
- implemented a transparent heuristic recommender suitable for limited assessment time,
- focused on correctness, explainability, and stable runtime behavior.

Challenge 3: oral-exam readiness
Assessment requires concise demonstration and technical defence within strict time.

Resolution:
- prepared end-to-end workflow with predictable outputs,
- aligned repository artifacts (README, API docs, report, slides outline) to examiner checklist.

8. Limitations

- Recommendation logic is heuristic and not user-personalized.
- Genre values are currently coarse due to source-field constraints.
- Ratings are source-scale dependent (0-10) without optional normalization toggle.
- Full production concerns (advanced caching, observability pipeline, role-based authorization) are not yet implemented.

9. Future Improvements

- Add optional rating normalization strategy (0-10 to 0-5).
- Add user profiles and interaction history for personalized recommendation.
- Add pagination metadata, richer filters, and sorting options.
- Add CI pipeline with coverage threshold and automated quality checks.
- Deploy production profile with PostgreSQL and structured monitoring.

10. Deployment and Operational Status

Deployment target and runtime stack

The deployment target is Tencent Cloud CVM (Ubuntu 22.04) with the following runtime setup:
- Uvicorn process managed by systemd (bookapi.service).
- Nginx reverse proxy on port 80 forwarding to 127.0.0.1:8000.
- SQLite-backed runtime for coursework demonstration.

Deployment validation performed on server

The following checks were successfully completed on the server host:
- bookapi.service running (active (running)).
- Uvicorn startup logs confirm successful application startup.
- Local app probe succeeded: curl http://127.0.0.1:8000/health -> HTTP 200.
- Reverse-proxy probe succeeded: curl http://127.0.0.1/health -> HTTP 200.
- Documentation endpoint probe succeeded: curl http://127.0.0.1/docs -> HTTP 200.

External connectivity issue and mitigation

During external access verification, repeated timeouts were observed from multiple client networks despite healthy in-host service checks.

Mitigation adopted for reliable oral-demo accessibility:
- A temporary reverse tunnel endpoint was established from the server to a public HTTPS URL.
- Public access was validated via:
  - /health
  - /docs

This ensured the project remained demonstrable end-to-end even when direct inbound path behavior was inconsistent.

Operational note for examiners

If a temporary tunnel URL is used, it may rotate between sessions and requires an active tunnel process. Core service reproducibility remains verifiable through repository setup steps and server-side health checks documented above.

11. Generative AI Declaration

Tools used

- GitHub Copilot
- ChatGPT

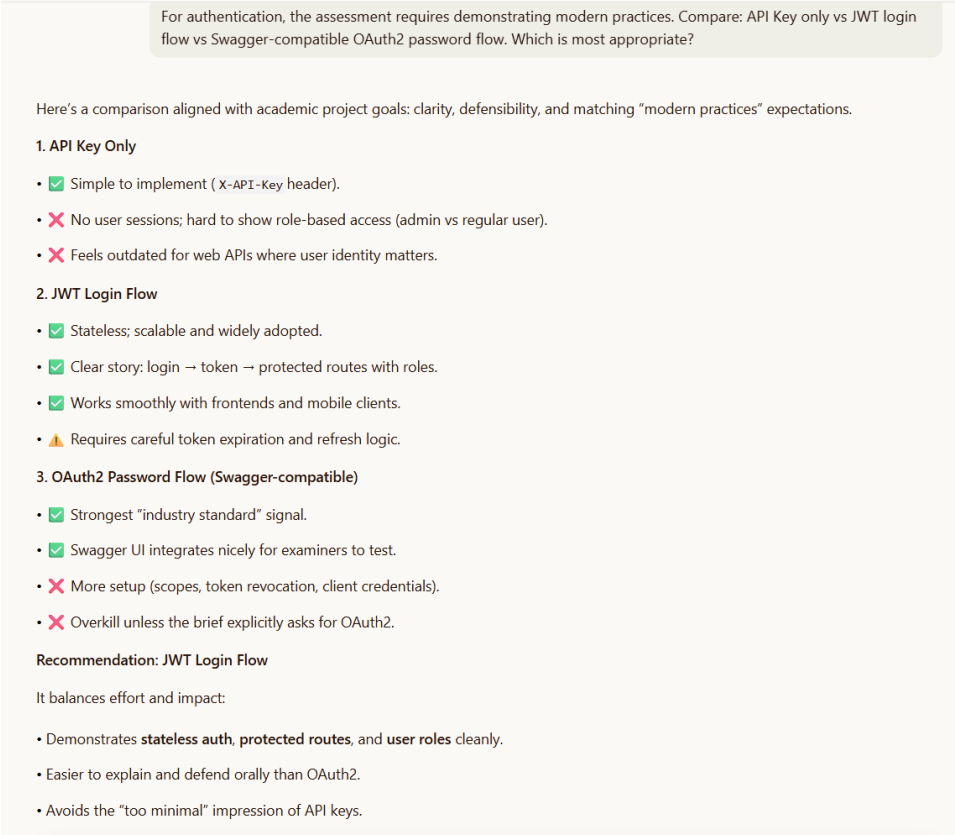
Purpose and level of use

- architecture and endpoint planning,
- high-level alternative exploration (auth and deployment options),
- debugging assistance and failure-path diagnosis,
- data-import script hardening for real dataset edge cases,
- documentation and presentation refinement.

The usage was systematic rather than ad-hoc: each major task followed an objective -> options -> decision -> validation cycle.

Examples of alternative exploration

- Authentication alternatives considered: API-key-only vs JWT token flow.
- Deployment alternatives considered: platform-managed hosting vs CVM runtime + fallback tunnel path.
- Data ingestion alternatives considered: strict parser vs resilient fallback parser pipeline.



For authentication, the assessment requires demonstrating modern practices. Compare: API Key only vs JWT login flow vs Swagger-compatible OAuth2 password flow. Which is most appropriate?

Here's a comparison aligned with academic project goals: clarity, defensibility, and matching "modern practices" expectations.

1. API Key Only

- ✅ Simple to implement (`X-API-Key` header).
- ❌ No user sessions; hard to show role-based access (admin vs regular user).
- ❌ Feels outdated for web APIs where user identity matters.

2. JWT Login Flow

- ✅ Stateless; scalable and widely adopted.
- ✅ Clear story: login → token → protected routes with roles.
- ✅ Works smoothly with frontends and mobile clients.
- ⚠️ Requires careful token expiration and refresh logic.

3. OAuth2 Password Flow (Swagger-compatible)

- ✅ Strongest "industry standard" signal.
- ✅ Swagger UI integrates nicely for examiners to test.
- ❌ More setup (scopes, token revocation, client credentials).
- ❌ Overkill unless the brief explicitly asks for OAuth2.

Recommendation: JWT Login Flow

It balances effort and impact:

- Demonstrates **stateless auth**, **protected routes**, and **user roles** cleanly.
- Easier to explain and defend orally than OAuth2.
- Avoids the "too minimal" impression of API keys.

Figure: An example of GenAI logs

Accepted/rejected choices were based on coursework criteria, explainability in oral defense, and runtime reliability.

Verification and control

- AI-generated suggestions were reviewed before adoption.
- Functional correctness was validated by endpoint execution and automated tests.
- Non-trivial implementation decisions (security, importer robustness, analytics interpretation) were manually verified and adjusted.

Evidence-backed workflow quality

- GenAI use contributed to solution exploration (not only code completion), including design trade-offs and contingency planning.
- Final implementation decisions, integration sequence, and submission packaging remained student-controlled.

Supplementary evidence of conversation excerpts

- A selected excerpt log is included in docs/GENAI_EXCERPTS.md (docs/GENAI_EXCERPTS.md).
- The excerpt file records planning, option comparison, implementation/debugging, deployment fallback decisions, and documentation stages.
- Raw exported JSONL-style snippets are included in docs/GENAI_EXPORTED_LOG_SNIPPETS.md (docs/GENAI_EXPORTED_LOG_SNIPPETS.md).
- Evidence packaging window used for submission: 2026-04-10 to 2026-04-20 (available interactions in this transcript are concentrated on 2026-04-19 to 2026-04-20).
- Full transcript history is retained in local editor storage for traceability.

Responsible use statement

Generative AI was used as an assistive productivity tool, not as an unverified code source. Final decisions, integration, testing, and submitted outputs were curated by the student.

12. References

Book Recommendation Dataset. *www.kaggle.com*. [Online]. Available from: https://www.kaggle.com/datasets/arashnic/book-recommendation-dataset.

pytest: Helps You Write Better Programs - Pytest Documentation. *docs.pytest.org*. [Online]. Available from: https://docs.pytest.org/.

SQLAlchemy Documentation - SQLAlchemy 2.0 Documentation. *docs.sqlalchemy.org*. [Online]. Available from: https://docs.sqlalchemy.org/.

FastAPI 2023. FastAPI. *fastapi.tiangolo.com*. [Online]. Available from: https://fastapi.tiangolo.com/.